MODULE 25 · EXAM STRATEGY - ALL BLOCKS

Practice paper 1

Thirty new questions in the exam's proportions: 7 from Block 1, 8 from Block 2, 7 from Block 3, 8 from Block 4.

QUESTIONS 30 TIMING about 80 s each MODULE 25 of 28

admin@sthithapragna.com

Treat it as the real thing

- **No paper.** Trace everything in your head, as you will on the day.
- Commit to an answer before each walkthrough. Pause the video if you need more time.
- Keep a tally of your misses by block. The scoring guide at the end uses it.
- Watching the answers without choosing first teaches almost nothing.

What is the output of this program?

```
1  a = 7.
2  b = "7"
3  c = 7e0
4  print(type(a) == type(c), type(b) == type(a))
```

A True False

B False False

C True True

D False True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = 7.
2  b = "7"
3  c = 7e0
4  print(type(a) == type(c), type(b) == type(a))
```

A **True False**
7. and 7e0 are both floats; "7" is a str.

B **False False**
A point or an E makes a float, so a and c match.

C **True True**
Quotes make a str, whatever the digits.

D **False True**
Both comparisons are reversed.

BLOCK 1

A point or an E makes a float. Quotes make a str.

How many stars does this program print?

```
for i in range(1, 10, 4):
    print("*", end="")
```

A Two

B Three

C Four

D Nine

◆ PAUSE AND DECIDE

How many stars does this program print?

```
for i in range(1, 10, 4):
    print("*", end="")
```

A Two

range(1, 10, 4) has three values: 1, 5 and 9.

B Three

1, 5, 9. The next would be 13, past the stop.

C Four

13 is past the stop of 10.

D Nine

The step is 4, not 1.

BLOCK 2

With a step other than 1, list the values; stop before reaching the stop.

What is the output of this program?

```
1  nums = [10, 20, 30, 40, 50]
2  print(nums[-1:1:-2])
```

A [50, 40, 30]

B []

C [50, 30]

D [50, 30, 10]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  nums = [10, 20, 30, 40, 50]
2  print(nums[-1:1:-2])
```

A `[50, 40, 30]`
The step is -2, so every second element.

B `[]`
A negative step can count down from -1 to 1.

C **`[50, 30]`**
Indexes 4 and 2. Index 1 is the stop, excluded.

D `[50, 30, 10]`
Index 0 is beyond the stop of 1.

BLOCK 3

Negative step: start included, stop excluded, every step-th element.

What is the output of this program?

```
def grade(n):
    if n >= 50:
        return "pass"
    elif n >= 40:
        return "close"

print(grade(45), grade(30))
```

A `close fail`

B `close None`

C `None None`

D The code raises a TypeError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def grade(n):
2      if n >= 50:
3          return "pass"
4      elif n >= 40:
5          return "close"
6
7  print(grade(45), grade(30))
```

A `close fail`

Nothing in the code produces the word fail.

B `close None`

45 returns close; 30 reaches no return, so None.

C `None None`

45 does reach the elif's return.

D The code raises a TypeError

A missing return is legal: it gives None.

BLOCK 4

Any path through a function without a return gives None.

What is the output of this program?

```
1  print(0o12 + 0b11)
```

A 23

B 15

C 1211

D 13

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(0o12 + 0b11)
```

A 23

Reads 0o12 as decimal 12.

B 15

Reads 0b11 as 5. Binary 11 is 3.

C 1211

Prefixed literals are ints, so + adds them.

D 13

Octal 12 is 10, binary 11 is 3.

BLOCK 1

Convert each prefixed literal with place values, then add normally.

What is the output of this program?

```
count = 0
for i in range(3):
    for j in range(i):
        count += 1
print(count)
```

A 9

B 6

C 3

D 0

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  count = 0
2  for i in range(3):
3      for j in range(i):
4          count += 1
5  print(count)
```

A 9

That would need range(3) for the inner loop as well.

B 6

Adds 0 + 1 + 2 + 3: the outer range stops at 2.

C 3

Inner passes: 0, then 1, then 2.

D 0

The inner loop runs when i is 1 and 2.

BLOCK 2

Inner range uses the outer variable: add each pass, do not multiply.

> Each name is considered on its own.

Which two are legal variable names? (Choose two.)

A `pass_`

B `for`

C `None`

D `nOne`

◆ PAUSE AND DECIDE

Which two are legal variable names? (Choose two.)

A `pass_`

A keyword with an underscore is a different name.

B `for`

for is a keyword.

C `None`

None is a keyword, capitalised.

D `nOne`

Case matters: nOne is not None.

BLOCK 1

Check the exact spelling against the 35 keywords, case included.

What is the output of this program?

```
try:
    print(int("abc"))
except (TypeError, IndexError):
    print("A")
except ArithmeticError:
    print("B")
except Exception:
    print("C")
```

A. A

B. B

C. C

D. The program stops with an unhandled ValueError

◆ PAUSE AND DECIDE

What is the output of this program?

```
try:
    print(int("abc"))
except (TypeError, IndexError):
    print("A")
except ArithmeticError:
    print("B")
except Exception:
    print("C")
```

A. A
int("abc") raises ValueError, which is not in the tuple.

B. B
ValueError is not an ArithmeticError.

C. C
ValueError falls through to the Exception branch.

D. The program stops with an unhandled ValueError
Exception catches every everyday error.

BLOCK 4

Name the exception, then take the first branch that matches it or a parent.

What is the output of this program?

```
1  text = "Hello World"
2  print(text.lower().count("o"), text.find("World"))
```

A 2 6

B 1 6

C 2 7

D 2 -1

◆ PAUSE AND DECIDE

What is the output of this program?

```
1 text = "Hello World"
2 print(text.lower().count("o"), text.find("World"))
```

A `2 6`

Two o's after lowering; World starts at index 6.

B `1 6`

lower() keeps both o's: hello world.

C `2 7`

Indexes start at 0, so W is at 6.

D `2 -1`

find searches the original text, which contains World.

BLOCK 3

Count from 0. find returns the index of the first match, or -1.

What is the output of this program?

```
n = 1
while n < 50:
    n *= 3
    if n == 27:
        break
else:
    print("done")
print(n)
```

A done, then 27

B 81

C done, then 81

D 27

◆ PAUSE AND DECIDE

What is the output of this program?

```
n = 1
while n < 50:
    n *= 3
    if n == 27:
        break
else:
    print("done")
print(n)
```

A `done, then 27`
break skips the loop's else.

B `81`
break stops the loop at 27.

C `done, then 81`
Ignores the break entirely.

D **27**
n: 3, 9, 27, then break. The else is skipped.

BLOCK 2

A loop's else runs only when no break fired.

What is the output of this program?

```
x = 10
def change():
    global x
    x = x // 3
    return x * 2

y = change()
print(x, y)
```

A 10 6

B 3 6

C 3 3

D The code raises an UnboundLocalError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 10
2  def change():
3      global x
4      x = x // 3
5      return x * 2
6
7  y = change()
8  print(x, y)
```

A `10 6`

global x makes the function change the outer x.

B `3 6`

x becomes 10 // 3, which is 3; then 3 * 2 is returned.

C `3 3`

*The function returns x * 2.*

D The code raises an UnboundLocalError

global x makes x the global, so it has a value.

BLOCK 4

With global, assigning inside a function changes the outer name.

What is the output of this program?

```
1  print(-9 // 4, -9 % 4)
```

A -2 -1

B -3 -3

C -3 3

D -2 1

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(-9 // 4, -9 % 4)
```

A `-2 -1`

Truncates towards zero, like C. Python floors.

B `-3 -3`

% takes the sign of the divisor, 4.

C `-3 3`

−2.25 floors to −3; −3 * 4 + 3 is −9.

D `-2 1`

Neither half follows Python's floor rule.

BLOCK 1

// rounds towards minus infinity; % takes the sign of the divisor.

What is the output of this program?

```
1  a = ["x"]
2  a.extend("yz")
3  a.append("yz")
4  a.insert(1, 0)
5  print(len(a), a[1])
```

A 4 0

B 5 y

C 3 0

D 5 0

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = ["x"]
2  a.extend("yz")
3  a.append("yz")
4  a.insert(1, 0)
5  print(len(a), a[1])
```

A `4 0`

extend("yz") adds two elements, not one.

B `5 y`

insert(1, 0) puts 0 at index 1.

C `3 0`

append adds one more element, and insert another.

D `5 0`

x, y, z, then "yz", then 0 inserted at index 1.

BLOCK 3

extend adds each item of a sequence, even a string's characters. append adds one.

What is the output of this program?

```
i = 0
total = 0
while i < 6:
    i += 1
    if i % 3 == 0:
        continue
    total += i
print(total)
```

A 21

B 12

C 15

D 9

◆ PAUSE AND DECIDE

What is the output of this program?

```
i = 0
total = 0
while i < 6:
    i += 1
    if i % 3 == 0:
        continue
    total += i
print(total)
```

A 21

Ignores the continue.

B 12

1 + 2 + 4 + 5: the multiples of 3 are skipped.

C 15

Includes 3, or leaves out 6, but not both.

D 9

Skips more than the multiples of 3.

BLOCK 2

continue skips the rest of the pass. Check the update happens before it.

What is the output of this program?

```
def f(n):
    if n < 10:
        return n
    return f(n // 10) + n % 10

print(f(905))
```

A 14

B 509

C 5

D 905

◆ PAUSE AND DECIDE

What is the output of this program?

```
def f(n):
    if n < 10:
        return n
    return f(n // 10) + n % 10

print(f(905))
```

A 14

9 + 0 + 5: the digits of 905, summed.

B 509

Builds the reverse instead of a sum.

C 5

Stops after one step. The base case is reached later.

D 905

Nothing returns n unchanged unless n < 10.

BLOCK 4

Trace calls going down, then add the returns coming back up.

What is the output of this program?

```
1  print(2 + 3 * 2 ** 2 - 4 // 3)
```

A 12

B 21

C 13.0

D 13

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(2 + 3 * 2 ** 2 - 4 // 3)
```

A `12`
4 // 3 is 1, not 2.

B `21`
*Does + before * somewhere. Power, then * and //, then + and -.*

C `13.0`
No / and no float: the result is an int.

D **`13`**
2 ** 2 is 4; 3 * 4 is 12; 4 // 3 is 1; 2 + 12 - 1.

BLOCK 1

Power, then * / // %, then + -. Then ask whether it is an int or a float.

What is the output of this program?

```
1  t = (1, 2) * 2
2  print(t[1:3], len(t))
```

A (2, 1) 4

B [2, 1] 4

C (2, 1) 2

D (2,) 4

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  t = (1, 2) * 2
2  print(t[1:3], len(t))
```

A `(2, 1) 4`

(1, 2, 1, 2): slice indexes 1 and 2.

B `[2, 1] 4`

Slicing a tuple gives a tuple.

C `(2, 1) 2`

** 2 repeats the elements: the tuple has 4.*

D `(2,) 4`

The slice stops before index 3, so two elements.

BLOCK 3

A slice has the same type as what you sliced.

What is the output of this program?

```
x = 15
if x % 3 == 0:
    print("Fizz", end="")
if x % 5 == 0:
    print("Buzz", end="")
elif x > 10:
    print("Big", end="")
else:
    print("-", end="")
```

A Fizz

B FizzBuzzBig

C FizzBuzz

D Buzz

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 15
2  if x % 3 == 0:
3      print("Fizz", end="")
4  if x % 5 == 0:
5      print("Buzz", end="")
6  elif x > 10:
7      print("Big", end="")
8  else:
9      print("-", end="")
```

A `Fizz`

The second if is a separate decision.

B `FizzBuzzBig`

Big is in the same chain as Buzz, so it is skipped.

C **`FizzBuzz`**

Two decisions: Fizz from the first, Buzz ends the second chain.

D `Buzz`

15 % 3 == 0, so Fizz prints first.

BLOCK 2

Every if that is not an elif starts a new decision.

What happens when this program runs?

```
1  def area(w, h=2):
2      return w * h
3
4  print(area(3), area(h=4, w=1), area(2, w=5))
```

A 6 4 10 is printed

B 6 4 is printed, then a TypeError

C Nothing is printed; the program raises a TypeError

D Nothing is printed; the program raises a SyntaxError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  def area(w, h=2):
2      return w * h
3
4  print(area(3), area(h=4, w=1), area(2, w=5))
```

A 6 4 10 is printed
area(2, w=5) fills w twice.

B 6 4 is printed, then a TypeError
print evaluates every argument before printing anything.

C Nothing is printed; the program raises a TypeError
The third call fails before print can run.

D Nothing is printed; the program raises a SyntaxError
No positional follows a keyword here, so it compiles.

BLOCK 4

print evaluates all its arguments first. One failing argument means no output from that print.

> d = {"a": 1, "b": 2} has been assigned.

Which two expressions evaluate to True? (Choose two.)

A `1 in d`

B `"b" in d`

C `d.get("c", 3) == 3`

D `len(d.items()) == 4`

◆ PAUSE AND DECIDE

Which two expressions evaluate to True? (Choose two.)

A `1 in d`

in checks keys. 1 is a value.

B `"b" in d`

b is a key.

C `d.get("c", 3) == 3`

c is missing, so get returns the default 3.

D `len(d.items()) == 4`

Two pairs, so 2.

BLOCK 3

in checks keys. get returns its default only when the key is missing.

What is the output of this program?

```
1  print(1, 2, sep="+", end="=")
2  print(3)
```

A `1 + 2 = 3`

B `1+2+=3`

C 1+2=, then 3 on a new line

D `1+2=3`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(1, 2, sep="+", end="=")
2  print(3)
```

A `1 + 2 = 3`

sep replaces the space, so there are no spaces.

B `1+2+=3`

sep goes between arguments only, never after the last.

C `1+2=`, then `3` on a new line

end replaced the newline, so 3 follows on the same line.

D **`1+2=3`**

sep between, end at the end, then 3 continues the line.

BLOCK 1

sep only between arguments; end replaces the newline.

What is the output of this program?

```
for i in range(2, 8, 3):
    pass
print(i)
```

A 8

B 7

C 5

D 2

◆ PAUSE AND DECIDE

What is the output of this program?

```
for i in range(2, 8, 3):
    pass
print(i)
```

A 8

range never produces its stop.

B 7

7 is not a value of this range.

C 5

2, then 5. The loop variable keeps its last value.

D 2

The loop runs twice, so i moves on to 5.

BLOCK 2

After a for loop, the variable holds the last value produced.

What is the output of this program?

```
def evens(limit):
    for n in range(limit):
        if n % 2 == 0:
            yield n * n

print(list(evens(5)))
```

A [0, 2, 4]

B [0, 4, 16]

C [4, 16]

D [0, 4, 16, 36]

◆ PAUSE AND DECIDE

What is the output of this program?

```
def evens(limit):
    for n in range(limit):
        if n % 2 == 0:
            yield n * n

print(list(evens(5)))
```

A `[0, 2, 4]`
*The generator yields n * n, not n.*

B **`[0, 4, 16]`**
Even n in range(5) are 0, 2, 4; squared.

C `[4, 16]`
*0 is even, and 0 * 0 is 0.*

D `[0, 4, 16, 36]`
range(5) stops at 4.

BLOCK 4

Trace the loop inside a generator; each yield adds one value.

What is the output of this program?

```
1  words = ["apple", "fig", "kiwi"]
2  print([w[0] for w in words if len(w) > 3])
```

A `['a', 'k']`

B `['a', 'f', 'k']`

C `['apple', 'kiwi']`

D `['f']`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  words = ["apple", "fig", "kiwi"]
2  print([w[0] for w in words if len(w) > 3])
```

A `['a', 'k']`

apple and kiwi pass the filter; w[0] takes their first letters.

B `['a', 'f', 'k']`

fig has 3 letters, so it is filtered out.

C `['apple', 'kiwi']`

The expression is w[0], not w.

D `['f']`

The test is len(w) > 3, not len(w) == 3.

BLOCK 3

Read a comprehension as: sequence, then filter, then expression.

What is the output of this program?

```
n = 1000
c = 0
while n > 0:
    c += 1
    n //= 10
print(c)
```

A 3

B 4

C 1000

D 1

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  n = 1000
2  c = 0
3  while n > 0:
4      c += 1
5      n //= 10
6  print(c)
```

A 3

Counts zeros. The loop counts every digit.

B 4

1000, 100, 10, 1: four passes.

C 1000

c counts passes; n is not printed.

D 1

The loop runs until n reaches 0.

BLOCK 2

n //= 10 drops one digit per pass, so counting passes counts digits.

What is the output of this program?

```
def add(item, box=None):
    if box is None:
        box = []
    box.append(item)
    return box

a = add(1)
b = add(2)
c = add(3, a)
print(a, b, c is a)
```

A [1] [2] False

B [1, 2, 3] [1, 2] True

C [1, 3] [2] False

D [1, 3] [2] True

◆ PAUSE AND DECIDE

What is the output of this program?

```
def add(item, box=None):
    if box is None:
        box = []
    box.append(item)
    return box

a = add(1)
b = add(2)
c = add(3, a)
print(a, b, c is a)
```

A `[1] [2] False`
c = add(3, a) appends to a itself.

B `[1, 2, 3] [1, 2] True`
None as the default makes a fresh list each time.

C `[1, 3] [2] False`
add returns the list it was given, so c is a.

D **`[1, 3] [2] True`**
Fresh lists for a and b; the third call extends a.

BLOCK 4

The None default gives a fresh list per call. A list passed in is changed in place.

What is the output of this program?

```
1  print(12 & 10 | 1, 5 ^ 1, ~4)
```

A 8 4 -5

B 9 6 -4

C 9 4 -4

D 9 4 -5

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  print(12 & 10 | 1, 5 ^ 1, ~4)
```

A `8 4 -5`

| then sets the last bit: 8 | 1 is 9.

B `9 6 -4`

5 ^ 1 clears the last bit: 4. And ~4 is -5.

C `9 4 -4`

~x is -x - 1, so ~4 is -5.

D `9 4 -5`

(12 & 10) | 1 is 9; 101 ^ 001 is 100; -4 - 1.

BLOCK 1

& before ^ before |. And ~x is -x - 1.

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a
3  b = b + [4]
4  a[0] = 9
5  print(a, b)
```

A `[9, 2, 3] [9, 2, 3, 4]`

B `[9, 2, 3] [1, 2, 3, 4]`

C `[9, 2, 3, 4] [9, 2, 3, 4]`

D `[1, 2, 3] [1, 2, 3, 4]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a
3  b = b + [4]
4  a[0] = 9
5  print(a, b)
```

A `[9, 2, 3] [9, 2, 3, 4]`
b = b + [4] made b a new list before a changed.

B **`[9, 2, 3] [1, 2, 3, 4]`**
+ built a new list for b; a[0] = 9 changes a only.

C `[9, 2, 3, 4] [9, 2, 3, 4]`
+ does not extend the shared list.

D `[1, 2, 3] [1, 2, 3, 4]`
a[0] = 9 changes a.

BLOCK 3

b = b + [...] builds a new list. Only +=, methods and index assignment change a shared one.

What is the output of this program?

```
for n in [4, 6, 9, 10]:
    if n % 3 == 0 and n % 2 == 1:
        print("found", n)
        break
else:
    print("none")
```

A found 9

B found 6

C none

D found 9, then none

◆ PAUSE AND DECIDE

What is the output of this program?

```
for n in [4, 6, 9, 10]:
    if n % 3 == 0 and n % 2 == 1:
        print("found", n)
        break
else:
    print("none")
```

A `found 9`

The first odd multiple of 3 in the list.

B `found 6`

6 is a multiple of 3, but even.

C `none`

9 matches, so break fires and the else is skipped.

D found 9, then none

break skips the loop's else.

BLOCK 2

and needs both sides true. A break skips the loop's else.

> Consider Python's built-in exception hierarchy.

Which two exceptions are subclasses of LookupError? (Choose two.)

A `KeyError`

B `ValueError`

C `ZeroDivisionError`

D `IndexError`

◆ PAUSE AND DECIDE

Which two exceptions are subclasses of LookupError? (Choose two.)

A `KeyError`
A missing key is a lookup problem.

B `ValueError`
ValueError sits directly under Exception.

C `ZeroDivisionError`
That is an ArithmeticError.

D `IndexError`
A bad position is a lookup problem.

BLOCK 4

LookupError: KeyError and IndexError. ArithmeticError: ZeroDivisionError, OverflowError.

How did you do?

SCORE OUT OF 30	WHAT IT MEANS	WHAT TO DO NEXT
27 to 30	comfortably above the pass mark	take paper 2 within a few days
21 to 26	at or above 70%, the pass mark	revisit the block where most misses fell, then paper 2
Below 21	below the pass mark	rework the weakest block's modules before paper 2

ON THE EXAM

21 out of 30 is 70%. Aim for a margin above it, not for the line itself.

Find the block, then the modules

BLOCK	QUESTIONS IN THIS PAPER	MODULES TO REVISIT
1: Fundamentals	1, 5, 7, 12, 16, 21, 27	1 to 5
2: Control flow	2, 6, 10, 14, 18, 22, 25, 29	6 to 10
3: Data collections	3, 9, 13, 17, 20, 24, 28	11 to 15
4: Functions and exceptions	4, 8, 11, 15, 19, 23, 26, 30	16 to 21

WHAT MATTERS MOST

Where your misses cluster matters more than the total. Three misses in one block is a pattern; spread out, it is noise.